

Email Spam Classifier

Name : Hari P

Project Type: Individual Project

EXECUTIVE SUMMARY:

Email spam is a growing problem that causes time loss, security risks, and privacy issues. This project develops a machine learning-based system to classify emails as Spam or Ham. Natural Language Processing (NLP) techniques are used to analyze email text. TF-IDF vectorization converts text into numerical features for model training. Naive Bayes, Logistic Regression, and SVM algorithms are implemented and compared. Among them, SVM delivers the best performance and is suitable for real-time use.

PROJECT OVERVIEW:

Objective:

The objective of this project is to preprocess email text and convert it into numerical features using TF-IDF. Multiple machine learning models are trained and evaluated using accuracy and classification metrics. The goal is to develop a reliable system for detecting spam emails.

Scope:

- Text-based spam detection
- NLP preprocessing using TF-IDF
- Comparison of multiple ML algorithms
- Performance evaluation using standard metrics
- Visualization using confusion matrix
- Real-time prediction function

Key Features:

- Detects whether an email is spam or legitimate (ham)
- Uses text processing to extract meaningful features from emails
- Lightweight and fast — can be integrated into existing email systems
- Real-time classification of emails
- High accuracy with low false-positive rate
- Option to visualize important spam-indicating keywords

TECHNICAL IMPLEMENTATION:

Architecture:

- Email text is taken as input from the dataset or user.
- Text preprocessing is performed to clean and prepare the data.
- TF-IDF vectorization converts text into numerical feature vectors.
- Machine learning models (Naive Bayes, Logistic Regression, SVM) process the features.
- The trained model classifies the email as Spam or Ham.
- Results are evaluated using accuracy, classification report, and confusion matrix.

Technology Stack:

- **Programming Language:** Python
- **Data Handling:** pandas, NumPy
- **Machine Learning:** scikit-learn
- **Text Processing (NLP):** TF-IDF Vectorizer
- **Algorithms Used:** Naive Bayes, Logistic Regression, Support Vector Machine (SVM)
- **Visualization:** Matplotlib, Seaborn
- **Dataset:** UCI SMS Spam Collection / Kaggle Spam Dataset

Performance Matrices:

- Accuracy: Measures the overall correctness of the model's predictions.
- Precision: Indicates how many emails predicted as spam are actually spam.
- Recall: Measures how many actual spam emails are correctly identified.
- F1-Score: Provides a balance between precision and recall.
- Confusion Matrix: Shows true positives, true negatives, false positives, and false negatives visually.

SOURCE CODE IMPLEMENTATION (Screenshot):

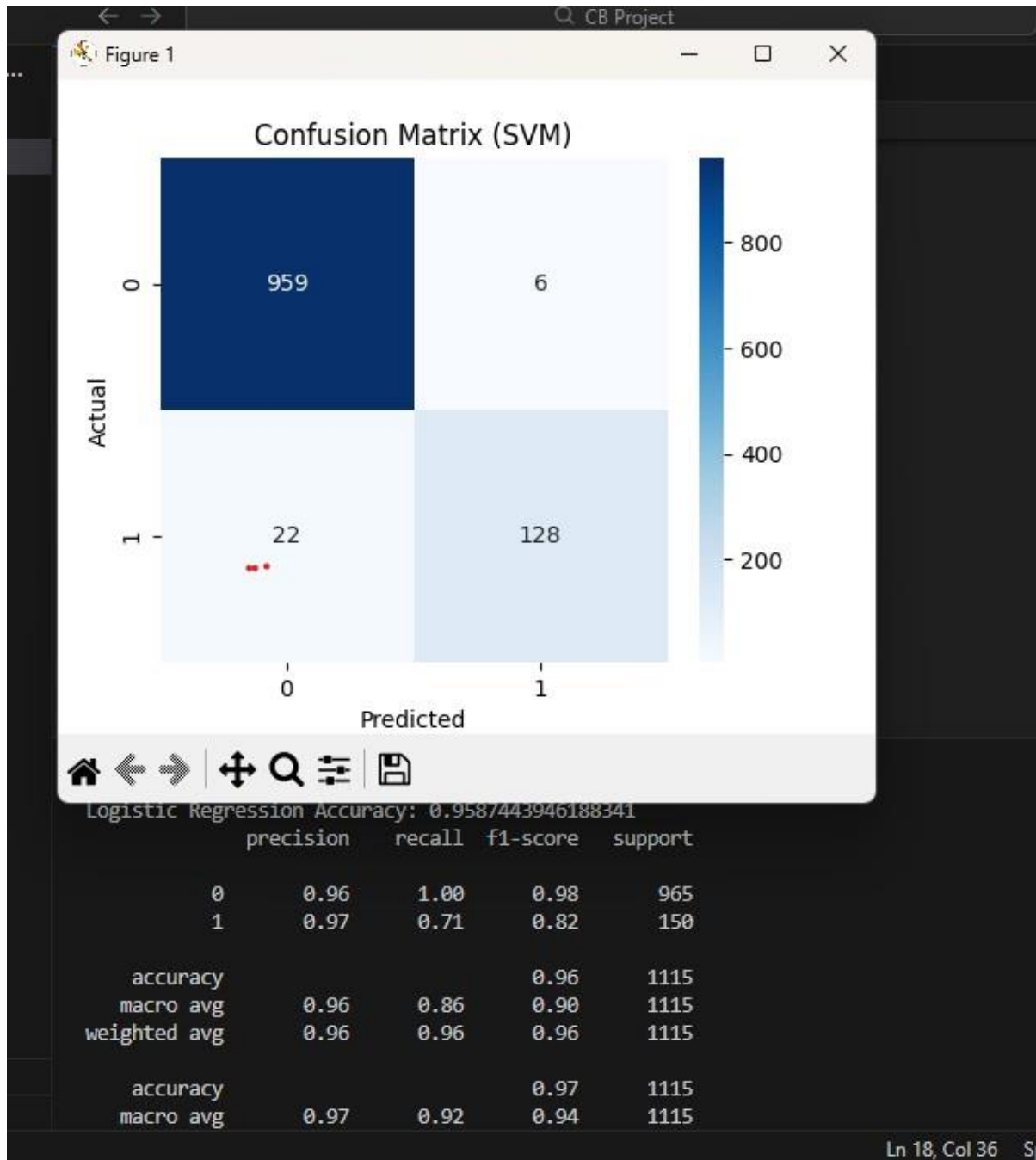
```

Email spam > ...
1  import pandas as pd
2  import numpy as np
3  import matplotlib.pyplot as plt
4  import seaborn as sns
5
6  from sklearn.model_selection import train_test_split
7  from sklearn.feature_extraction.text import TfidfVectorizer
8  from sklearn.naive_bayes import MultinomialNB
9  from sklearn.linear_model import LogisticRegression
10 from sklearn.svm import LinearSVC
11 from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
12
13 # Load dataset
14 data = pd.read_csv("spam.csv", encoding='latin-1')
15
16 # Keep only required columns
17 data = data[['v1', 'v2']]
18 data.columns = ['label', 'message']
19
20 print(data.head())
21
22 # Convert labels to binary
23 data['label'] = data['label'].map({'ham': 0, 'spam': 1})
24
25 # Train-test split
26 X_train, X_test, y_train, y_test = train_test_split(
27     data['message'],
28     data['label'],
29     test_size=0.2,
30     random_state=42
31 )
32
```

```
Email spam > ...
33 vectorizer = TfidfVectorizer(
34     stop_words='english',
35     max_features=5000
36 )
37
38 X_train_tfidf = vectorizer.fit_transform(X_train)
39 X_test_tfidf = vectorizer.transform(X_test)
40
41 nb_model = MultinomialNB()
42 nb_model.fit(X_train_tfidf, y_train)
43
44 nb_pred = nb_model.predict(X_test_tfidf)
45
46 print("Naive Bayes Accuracy:", accuracy_score(y_test, nb_pred))
47 print(classification_report(y_test, nb_pred))
48
49 lr_model = LogisticRegression(max_iter=1000)
50 lr_model.fit(X_train_tfidf, y_train)
51
52 lr_pred = lr_model.predict(X_test_tfidf)
53
54 print("Logistic Regression Accuracy:", accuracy_score(y_test, lr_pred))
55 print(classification_report(y_test, lr_pred))
56
57 svm_model = LinearSVC()
58 svm_model.fit(X_train_tfidf, y_train)
59
60 svm_pred = svm_model.predict(X_test_tfidf)
61
62 print("SVM Accuracy:", accuracy_score(y_test, svm_pred))
63 print(classification_report(y_test, svm_pred))
64
Ln 18, Col 36 Spaces: 4 UTF-8 CRU
```

```
Email spam > ...
56
57 svm_model = LinearSVC()
58 svm_model.fit(X_train_tfidf, y_train)
59
60 svm_pred = svm_model.predict(X_test_tfidf)
61
62 print("SVM Accuracy:", accuracy_score(y_test, svm_pred))
63 print(classification_report(y_test, svm_pred))
64
65 cm = confusion_matrix(y_test, svm_pred)
66
67 plt.figure(figsize=(5,4))
68 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
69 plt.xlabel("Predicted")
70 plt.ylabel("Actual")
71 plt.title("Confusion Matrix (SVM)")
72 plt.show()
73
74 def predict_email(text):
75     text_vector = vectorizer.transform([text])
76     prediction = svm_model.predict(text_vector)
77     return "SPAM" if prediction[0] == 1 else "HAM"
78
79 # Example
80 email = "Congratulations! You won a free gift card. Click now!"
81 print("Prediction:", predict_email(email))
```

Email Spam Classifier Result:



KEY ACHIEVEMENTS:

Technical Accomplishments:

- Successfully implemented end-to-end text preprocessing and feature extraction using TF-IDF vectorization.
- Trained and compared multiple machine learning models including Naive Bayes, Logistic Regression, and SVM.
- Achieved high classification accuracy, with SVM delivering the best overall performance.
- Evaluated models using standard performance metrics such as accuracy, precision, recall, F1-score, and confusion matrix.
- Developed a real-time prediction function to classify new email messages as spam or ham.

Process Automation Benefits:

- Reduces manual effort by automatically classifying emails as spam or legitimate.
- Improves accuracy and consistency in spam detection compared to manual filtering.

LIMITATION AND CONSTRAINTS:

Current Limitations:

- The model relies only on text content and does not analyze attachments, links, or sender behavior.
- Performance depends on the quality and size of the training dataset.
- The system may struggle with newly evolving spam patterns not seen during training.

Resource Constraints:

- Requires sufficient computational resources for training large datasets.
- Memory usage increases with higher TF-IDF feature dimensions.
- Real-time deployment may require optimized hardware for faster processing.

PROJECT IMPACT AND VALUE:

Immediate Benefits:

- Automatically filters spam emails, reducing inbox clutter.
- Enhances user security by detecting malicious or unwanted messages.
- Saves time by minimizing manual email screening.

Long-Term Potentials:

- Can be extended to detect phishing, scam, and fraudulent emails.
- Scalable for integration with enterprise-level email systems.
- Can be enhanced using advanced deep learning and AI techniques.

CONCLUSION:

This project successfully demonstrates the use of machine learning and natural language processing techniques to classify emails as spam or legitimate. By applying TF-IDF feature extraction and evaluating multiple algorithms, the Support Vector Machine model achieved the best performance. The system is efficient, scalable, and provides a strong foundation for future enhancements such as phishing detection and real-world email system integration.